

# Architecture & Implementation Standards

Layer-Agnostic Architecture · Tech Stack · Code Standards · Testing · Security

This document extends the architecture blueprint with the full technical implementation standards used across the frontend, backend, data, testing, and security layers. All layers follow SOLID principles and DRY patterns. Each section maps directly to a layer in the 7-layer architecture.

## TECHNOLOGY STACK

|                    |                |                  |           |
|--------------------|----------------|------------------|-----------|
| Next.js App Router | TypeScript     | React            | Shadcn UI |
| React Query        | Zustand        | Zod              | Axios     |
| Auth0              | Python FastAPI | Microsoft Fabric | Supabase  |
| Jest               | Playwright     | Tailwind CSS     | nuqs      |

## LAYER-TO-STACK MAPPING

|                            |                                                                                       |
|----------------------------|---------------------------------------------------------------------------------------|
| Layer 1 – Presentation     | Next.js App Router · React · Shadcn UI · Tailwind · nuqs · Auth0 SDK (PKCE)           |
| Layer 2 – API Gateway      | Next.js middleware · withRateLimit wrapper · JWT validation · route handlers          |
| Layer 3 – IAM Service      | Auth0 Organizations · RBAC (Global Admin / Org Admin / Org Member) · token enrichment |
| Layer 4 – Business Logic   | Python FastAPI service (/python_service/) · OpenAI · vector embeddings · Pydantic     |
| Layer 5 – DAL              | dataAccess.ts · apiClient.ts (Axios) · React Query hooks · Zod validation             |
| Layer 6 – Fabric Connector | Microsoft Fabric REST API · OneLake · Lakehouse · service principal via Key Vault     |
| Layer 7 – Platform         | Azure Service Bus · OpenTelemetry · Azure Key Vault · feature flags                   |

## SECTION 01

# Frontend Standards

The frontend is a Next.js 13+ App Router application with TypeScript throughout. React Server Components are the default; 'use client' is used only where Web API access is strictly necessary. All data fetching goes through React Query custom hooks.

### NEXT.JS APP ROUTER CONVENTIONS

|                   |                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------|
| Rendering default | React Server Components (RSC) – never add 'use client' unless required for Web API access |
| Data fetching     | Server components fetch directly; client components use React Query hooks only            |
| Client components | Wrap in Suspense with fallback · dynamic import non-critical components                   |
| URL state         | nuqs for all URL search parameter state – never useState for URL params                   |
| Images            | WebP format · explicit width/height · lazy loading via next/image                         |
| Web Vitals        | Optimise LCP, CLS, FID – measure with Lighthouse in CI pipeline                           |
| Routing           | File-based routing with dynamic segments · layout.tsx for shared shells                   |

### TYPESCRIPT STANDARDS

|                       |                                                                                      |
|-----------------------|--------------------------------------------------------------------------------------|
| Types over interfaces | Use `type` definitions instead of `interface` – consistent across all files          |
| Entity type files     | Dedicated entity.type.ts files per domain: user.type.ts, organization.type.ts        |
| No any / unknown      | Never use `any` or `unknown` – create specific types or use generics instead         |
| Generic types         | Leverage generics for reusable components and utility functions                      |
| Strict config         | Enable strict: true in tsconfig – maximum type safety enforced in CI                 |
| Enums                 | Avoid enums – use const maps instead (e.g. const ROLE = { ADMIN: 'admin' } as const) |
| Aux verbs             | Descriptive variable names: isLoading, hasError, canSubmit, shouldRefetch            |

### COMPONENT ORGANISATION

|                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SHARED COMPONENTS</b> <ul style="list-style-type: none"><li>- Shared components -&gt; /app/components/</li><li>- Used by 2+ pages or contexts</li><li>- Named exports only (no default exports)</li><li>- Always include data-testid attribute</li><li>- Light + dark mode considered in all styles</li><li>- Mobile-first Tailwind responsive classes</li></ul> | <b>SINGLE-USE COMPONENTS</b> <ul style="list-style-type: none"><li>- Single-use -&gt; /app/[page]/components/</li><li>- Co-located with the page that owns it</li><li>- Example: /app/org-admin/agent-bits/components/</li><li>- Move to /app/components/ when reused</li><li>- File structure: component, subcomponents, helpers, static content, types</li></ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## UI & STYLING RULES

|                          |                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------|
| <b>Component library</b> | Shadcn UI + Radix primitives – customise via <code>className</code> , never override internals   |
| <b>Styling</b>           | Tailwind CSS only – no inline styles, no CSS modules unless absolutely necessary                 |
| <b>Responsiveness</b>    | Mobile-first: base styles for mobile, sm/md/lg breakpoints for larger screens                    |
| <b>Dark mode</b>         | Every component must work in both light and dark mode – use Tailwind <code>dark:</code> variants |
| <b>Syntax</b>            | Declarative JSX · function keyword for pure functions · no unnecessary curly braces              |

# State Management & Data Access Layer

Server state is managed exclusively by React Query. Client state is managed by Zustand. All HTTP requests go through an Axios instance (apiClient.ts). The 'use' prefix is reserved for React Query hooks only – never for utility functions or Zustand.

## REACT QUERY – SERVER STATE

|                    |                                                                                  |
|--------------------|----------------------------------------------------------------------------------|
| Scope              | All server state – API calls, caching, background refetching, optimistic updates |
| Hook rules         | Hooks contain ONLY direct CRUD calls to dataAccess or apiClient – nothing else   |
| No side effects    | Hooks must not call other hooks, Zustand stores, forms, or useEffect internally  |
| Naming             | 'use' prefix reserved for React Query hooks only (useUsers, useDocuments, useAI) |
| Error handling     | All errors handled through React Query error states – no try/catch in components |
| No redundant hooks | Avoid useEffect, useMemo, useCallback – React Query replaces all of these        |
| Stale time         | Configure per-query staleTime and gcTime in queryClient.ts – no magic defaults   |

## ZUSTAND – CLIENT STATE

|                 |                                                                                          |
|-----------------|------------------------------------------------------------------------------------------|
| Scope           | UI state · user preferences · complex local state · state surviving page reloads         |
| No server state | Zustand never holds API response data – that belongs to React Query cache                |
| Persistence     | Use persist middleware for state that must survive page reloads (e.g. sidebar collapsed) |
| Location        | All stores in /stores/ folder – never co-located with components                         |
| No mixing       | Zustand stores must not import or call React Query hooks – strict separation             |

## DATA ACCESS LAYER STRUCTURE

```
lib/
  dataAccess.ts # Data access abstraction – all DB/API calls go here
  apiClient.ts # Axios instance · base URL · interceptors · auth headers
  queryClient.ts # React Query client · staleTime · retry · error boundary config
  hooks/ # React Query hooks only – no utilities or helpers here
    useUsers.ts # User queries and mutations
    useOrganizations.ts # Org-level operations
    useDocuments.ts # Document management
    useAI.ts # AI assistant and chat hooks
```

## HTTP CLIENT – AXIOS STANDARDS

|                       |                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| <b>Client</b>         | Axios only – never native <code>fetch()</code> anywhere in the codebase                                 |
| <b>Instance</b>       | Single Axios instance in <code>apiClient.ts</code> – base URL, timeout, default headers configured once |
| <b>Auth headers</b>   | Auth0 access token injected via request interceptor – never set manually per call                       |
| <b>Error handling</b> | Response interceptor normalises all error shapes before they reach React Query                          |
| <b>Retries</b>        | React Query handles retries – do not add retry logic inside Axios interceptors                          |

## SERVER ACTIONS & API ROUTES

|                       |                                                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Server actions</b> | Full CRUD implemented as server actions for all entity types – preferred over API routes                            |
| <b>Export pattern</b> | <code>export const GET = withRateLimit(async (req: NextRequest) =&gt; {})</code> – always wrap                      |
| <b>Validation</b>     | Zod <code>safeParse()</code> on all request bodies – never trust raw input                                          |
| <b>Error handling</b> | <code>formatApiError</code> and <code>logServerError</code> from <code>@/app/lib/errorHandler</code> on every route |
| <b>Auth check</b>     | Auth0 <code>auth()</code> called first in every protected route before any business logic                           |
| <b>Params</b>         | <code>new URL(req.url) + searchParams.get()</code> for query params · <code>req.json()</code> for body              |
| <b>Status codes</b>   | 200 success · 201 created · 400 validation · 401 unauthorized · 500 server error                                    |
| <b>Cache headers</b>  | public for unauthenticated GET · private for authenticated · no-store for mutations                                 |

## Backend Service, Roles & Subscription Tiers

The Python FastAPI service runs alongside Next.js and handles AI operations, vector embeddings, and document processing. Auth0 enforces three role levels across all tenants. Three subscription tiers gate feature access per organisation.

### PYTHON FASTAPI SERVICE

|               |                                                                                         |
|---------------|-----------------------------------------------------------------------------------------|
| Location      | /python_service/ – runs alongside Next.js via npm run dev:all                           |
| Design        | RESTful endpoints with OpenAPI documentation auto-generated                             |
| Validation    | Pydantic models for all request/response types – equivalent of Zod on the TS side       |
| AI operations | OpenAI integration · vector embeddings · semantic search · system prompt building       |
| Database      | Fabric Lakehouse via service principal · Supabase for real-time subscriptions (planned) |
| Tables        | organizations · users · documents · system_prompts · company_knowledge                  |
| Logging       | Structured JSON logs · ERROR / WARN / INFO / DEBUG levels · correlation ID propagation  |
| Documentation | Postman collection maintained for all endpoints – updated with every API change         |

### USER ROLES & PERMISSIONS

| Role         | Access Level | Capabilities                                                                         |
|--------------|--------------|--------------------------------------------------------------------------------------|
| Global Admin | System-wide  | AI settings · user management · system configuration · all organisations · audit log |
| Org Admin    | Org-scoped   | Document control · member access · org settings · system prompts · knowledge base    |
| Org Member   | Read + chat  | Document viewing · AI chat · basic platform access · personal preferences            |

### SUBSCRIPTION TIERS

| Tier | Price | Documents | Features                                                                            |
|------|-------|-----------|-------------------------------------------------------------------------------------|
| Free | –     | 10 docs   | business_basics · content_writer · basic support                                    |
| Pro  | TBD   | Unlimited | Enhanced features · advanced system prompts · priority support                      |
| Pro+ | TBD   | Unlimited | All AI assistants · workflow integration · custom integrations · advanced analytics |

ROLE-BASED ROUTING & AUTH PATTERNS

|                   |                                                                                   |
|-------------------|-----------------------------------------------------------------------------------|
| Dashboard routing | Different layout.tsx shells based on role – Global Admin / Org Admin / Member     |
| Permission checks | Centralised utility functions – never inline role checks in components            |
| Session handling  | Consistent across all admin types – Auth0 session with role claims                |
| Token enrichment  | IAM service adds tenant_id, role, subscription_tier to JWT before forwarding      |
| Metadata          | Subscription status and tier stored in Auth0 user metadata – updated via webhooks |

LOGGING STANDARDS

|                   |                                                                                    |
|-------------------|------------------------------------------------------------------------------------|
| Global Admin logs | System-wide changes · user management actions · cross-tenant events                |
| Org Admin logs    | Org-specific changes · document management · member activity · AI usage            |
| Format            | Structured JSON with: timestamp, level, correlation_id, tenant_id, user_id, action |
| Levels            | ERROR (failures) · WARN (degraded) · INFO (normal operations) · DEBUG (dev only)   |
| Audit trail       | All system changes and user actions tracked – immutable append-only log            |

# Testing & Quality Assurance

Testing uses Jest with React Testing Library for unit and integration tests, and Playwright for end-to-end tests. 80% minimum code coverage is enforced in CI. All components must have data-testid attributes for reliable test selection.

## TESTING STACK & COVERAGE REQUIREMENTS

|                    |                                                                           |
|--------------------|---------------------------------------------------------------------------|
| Unit / integration | Jest + React Testing Library – mirrors source code directory structure    |
| End-to-end         | Playwright – full user workflow coverage across all three roles           |
| Coverage minimum   | 80% code coverage required – CI blocks merge if below threshold           |
| CI/CD integration  | All tests run on every PR – no merge without passing tests and coverage   |
| Mock strategy      | Complete mocking of external dependencies (Auth0, Fabric, OpenAI, Axios)  |
| Mock location      | Centralised mock configuration – no scattered inline mocks in test files  |
| Test isolation     | Each test runs independently with proper setup/teardown – no shared state |

## TEST COVERAGE AREAS

|                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>UNIT / INTEGRATION</b> <ul style="list-style-type: none"><li>- API route: auth, validation, errors, format</li><li>- Component: rendering, forms, state, a11y</li><li>- Hooks: CRUD operations, error states</li><li>- Stores: state mutations, persistence</li><li>- Utils: pure function input/output</li><li>- Permissions: role-based access checks</li></ul> | <b>E2E / PERFORMANCE</b> <ul style="list-style-type: none"><li>- E2E: full login flows for all 3 roles</li><li>- E2E: document CRUD workflows</li><li>- E2E: AI chat interactions</li><li>- E2E: subscription gating behaviour</li><li>- Performance: response time benchmarks</li><li>- Security: auth bypass attempts</li></ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## DATA-TESTID REQUIREMENTS

|                |                                                                                  |
|----------------|----------------------------------------------------------------------------------|
| Requirement    | Every React component must have a data-testid attribute – no exceptions          |
| Naming pattern | kebab-case descriptive names: data-testid='user-profile-card'                    |
| Granularity    | Page containers, forms, inputs, buttons, list items, modal triggers all need IDs |
| Playwright use | getByTestId() is the primary selector – avoids brittle CSS/text selectors        |
| Jest use       | screen.getByTestId() in React Testing Library – consistent across unit and E2E   |
| Review gate    | PRs without data-testid on new components are blocked in code review             |

## TESTING BEST PRACTICES

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| Arrange-Act-Assert | Every test follows AAA pattern – clear separation of setup, action, assertion |
| No implementation  | Test behaviour, not implementation – never assert on internal state           |
| Error scenarios    | Every happy path test must have a corresponding error/edge case test          |
| Async handling     | Use waitFor() and findBy* queries – never arbitrary setTimeout in tests       |
| Form testing       | userEvent over fireEvent – simulates real user interactions more accurately   |
| Snapshot policy    | Avoid snapshot tests for logic – use them only for stable UI components       |



# Security, Performance & Code Standards

Security is enforced at every layer – Auth0 at the edge, rate limiting on all routes, Zod/Pydantic on all inputs, and Fabric workspace isolation per tenant. SOLID and DRY principles apply throughout the entire codebase.

## SECURITY & COMPLIANCE

|                  |                                                                                       |
|------------------|---------------------------------------------------------------------------------------|
| Authentication   | Auth0 – PKCE flow on client · server-side session validation on every protected route |
| Rate limiting    | withRateLimit middleware wraps every API route handler – no unprotected endpoints     |
| Input validation | Zod safeParse() (TypeScript) · Pydantic (Python) on every request – never trust input |
| Tenant isolation | DAL scopes all queries by tenant_id · Fabric workspace isolation for enterprise tier  |
| Secrets          | Azure Key Vault for all credentials – zero hardcoded secrets in source or env files   |
| Audit trail      | Immutable structured logs for all system changes and user actions                     |
| Planned          | SOC2 compliance · advanced encryption · microservices for high-traffic scenarios      |

## PERFORMANCE OPTIMISATION

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| Server components  | Default to RSC – eliminates unnecessary client JS bundles and hydration cost      |
| Caching            | React Query staleTime per endpoint · HTTP cache headers (public/private/no-store) |
| Code splitting     | Dynamic imports for non-critical components – reduces initial bundle size         |
| Memory management  | Zustand stores cleaned up on unmount · React Query gc time configured per entity  |
| DB optimisation    | Query optimisation at DAL layer · connection pooling · parameterised queries only |
| Real-time          | WebSocket / SSE for live updates · automatic reconnection · message queuing       |
| Image optimisation | WebP format · next/image with explicit dimensions · lazy loading by default       |

## SOLID & DRY PRINCIPLES

|                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SOLID</b> <ul style="list-style-type: none"><li>- Single Responsibility: one reason to change per module</li><li>- Open/Closed: extend via composition, not modification</li><li>- Liskov: hooks and stores are interchangeable by contract</li><li>- Interface Segregation: small focused type contracts</li><li>- Dependency Inversion: depend on DAL interfaces not Fabric</li></ul> | <b>DRY</b> <ul style="list-style-type: none"><li>- No code duplication – extract shared logic to utils</li><li>- Shared components for anything used 2+ times</li><li>- Single Axios instance – never recreate per file</li><li>- Single queryClient – never instantiate per component</li><li>- Entity types defined once in entity.type.ts files</li></ul> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## CODE ORGANISATION PRINCIPLES

|                               |                                                                                         |
|-------------------------------|-----------------------------------------------------------------------------------------|
| <b>Separation of concerns</b> | Clear boundaries: presentation / business logic / data access / infrastructure          |
| <b>Type safety</b>            | TypeScript strict mode frontend · Python type hints throughout FastAPI service          |
| <b>Error handling</b>         | Consistent error shapes at every layer – no swallowed errors, no raw throws             |
| <b>File naming</b>            | Lowercase with dashes for directories · PascalCase for components · camelCase for utils |
| <b>Folder discipline</b>      | hooks/ = React Query only · stores/ = Zustand only · never mix concerns                 |
| <b>Security by default</b>    | Authentication required for all protected routes – no opt-in security                   |
| <b>Scalability</b>            | Multi-tenant architecture with org isolation baked in from day one                      |

## PLANNED ROADMAP

|                          |                                                                                           |
|--------------------------|-------------------------------------------------------------------------------------------|
| <b>Payments</b>          | TBD provider · subscription management · webhooks updating Auth0 metadata · checkout flow |
| <b>Real-time DB</b>      | Supabase real-time subscriptions for live document and notification updates               |
| <b>Advanced security</b> | SOC2 compliance · advanced encryption at rest and in transit                              |
| <b>Microservices</b>     | Decompose Python FastAPI into domain microservices for high-traffic scalability           |
| <b>Analytics</b>         | Advanced analytics dashboard for Pro+ tier · usage tracking · AI insight reporting        |